

Plataformas de programación web en entorno servidor. Aplicaciones LAMP.



Caso práctico



En la empresa **BK Programación** están a punto de hacerse cargo de un importante proyecto. **Ada**, la directora, va a comprometerse con un cliente y amigo, **Esteban**, que necesita ayuda con un problema concreto.

Esteban fundó hace ahora tres años **una pequeña empresa** dedicada a la venta de material tecnológico: cámaras, televisores, material informático, etc. Con el tiempo, esa pequeña empresa ha crecido. El número de ventas ha aumentado, así como el catálogo de productos que ofrece, e incluso ha abierto dos nuevas tiendas en localidades cercanas.

Pero como sucede muchas veces, **al aumentar el negocio** han ido surgiendo ciertas **necesidades**. El proyecto que Esteban le ha propuesto a Ada consiste en **desarrollar una web**. No una página web que explique dónde está la empresa o qué hace. Quiere una web enfocada a dos temas concretos: mejorar la **comunicación con sus clientes**, y que le aporte **información interna** a la empresa sobre su negocio.

Por ejemplo, quiere que los clientes puedan ver desde su casa los productos que vende, el precio de los mismos, o la disponibilidad en una u otra tienda. O que los empleados de la propia empresa puedan ver de forma sencilla el stock que tienen de los productos en las distintas sucursales, para poder decidir mejor qué productos se piden a los distribuidores y en qué cantidad.

Sin embargo, tal y como Ada ya le ha comentado a Esteban, la experiencia de BK Programación en el desarrollo de aplicaciones web es muy reducida. La mayoría de proyectos que han realizado hasta el momento se han centrado en aplicaciones para plataformas Windows y Linux, o para dispositivos móviles. Sólo uno de sus empleados, **Juan**, tiene cierta experiencia con la **programación web**.

Aun así, Esteban confía en que Ada y su empresa lograrán llevar a cabo el proyecto. No tiene prisa por ponerlo en funcionamiento, y sabe que a BK Programación le servirá para ir formando a sus empleados en temas relacionados en el desarrollo de aplicaciones web, por lo que finalmente llegan a un acuerdo.

En esta unidad se introducen los conceptos fundamentales en los que se basa el módulo. Se explica el concepto de ejecución de aplicaciones en entorno servidor, los componentes

implicados, y la diferencia con las páginas web estáticas y con la ejecución de código en el navegador web.

Se analizan las principales tecnologías disponibles para el desarrollo de aplicaciones web, y los conceptos fundamentales de cada una.

Pues, vayamos a ello.



[Ministerio de Educación y Formación Profesional](#) (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

[Aviso Legal](#)

1.- Características de la programación web.



Caso práctico



El nuevo proyecto al que se enfrenta **BK Programación** requiere que sus **empleados actualicen su formación** en temas relacionados con la **programación web**. **Juan**, el único con cierta experiencia en ese ámbito, ha **propuesto** a sus compañeros la realización de unas **jornadas** en las que explique a los demás los **fundamentos** del **desarrollo** de aplicaciones para la **web**. De esta forma, aquellos que quieran pasarán a formar parte del equipo que se dedique al nuevo proyecto.

Todos se apuntan a la **propuesta** de Juan, **incluyendo a Ada**, la directora.

El **primer objetivo** de las jornadas es ver en qué consiste la programación web. Por ejemplo, ver la **diferencia** existente entre hacer una **página web** y **programar** una **aplicación web**.

Seguro que **ya sabes** exactamente qué es una **página web**, e incluso conozcas cuáles son los pasos que se suceden para que, cuando visitas una web poniendo su dirección en el navegador, la página se descargue a tu equipo y se pueda mostrar. Sin embargo, este procedimiento que puede parecer sencillo, a veces no lo es tanto. Todo depende de cómo se haya hecho esa página.

Cuando una **página web se descarga** a tu ordenador, su contenido define qué se debe mostrar en pantalla. Este contenido está programado en un **lenguaje de marcado**, formado por etiquetas, que puede ser **HTML** o **XHTML**. Las etiquetas que componen la página indican el objetivo de cada una de las partes que la componen. Así, dentro de estos lenguajes hay etiquetas para indicar que un texto es un encabezado, que forma parte de una tabla, o que simplemente es un párrafo de texto.

Además, si la página está bien estructurada, la información que le indica al navegador el **estilo** con que se debe **mostrar cada parte de la página** estará almacenado en otro fichero, una  **hoja de estilos o CSS**. La hoja de estilos se encuentra indicada en la página web y el navegador la descarga junto a ésta. En ella nos podemos encontrar, por ejemplo, estilos que indican que el encabezado debe ir con tipo de letra Arial y en color rojo, o que los párrafos deben ir alineados a la izquierda.

Estos dos ficheros se descargan a tu ordenador desde un servidor web como respuesta a una petición. El proceso es el que se refleja en la siguiente figura.



Ilustración realizada con Dia (Elaboración propia)

Los pasos son los siguientes:

- 1.- **Un ordenador solicita a un servidor web una página** con extensión **.htm**, **.html** o **.xhtml**.
- 2.- **El servidor busca esa página** en un almacén de páginas (cada una suele ser un fichero).
- 3.- Si **el servidor encuentra esa página**, la recupera.
- 4.- Y por último **se la envía al navegador** para que éste pueda mostrar su contenido.

Este es un ejemplo típico de una **comunicación cliente-servidor**. El cliente es el que hace la petición e inicia la comunicación, y el servidor es el que recibe la petición y la atiende. En nuestro caso, el navegador es el cliente web.



Autoevaluación

¿Podemos ver una página web sin que intervenga un servidor web?

- ☐ Sí.
- ☐ No.

Efectivamente. Podemos ver páginas web con extensión **.htm**, **.html** o **.xhtml** que tengamos almacenadas en nuestro equipo simplemente abriéndolas con el navegador. En este caso la única utilidad del servidor web es enviar la página que solicitemos a nuestro equipo.

Piénsalo bien... ¿Cuál es realmente la utilidad del servidor web, según lo que acabamos de ver?

Solución

1. Opción correcta
2. Incorrecto

1.1.- Páginas web estáticas y dinámicas (I).

Las páginas que viste en el ejemplo anterior se llaman **páginas web estáticas**. Estas páginas se encuentran almacenadas en su forma definitiva, tal y como se crearon, y su contenido no varía. Son útiles para mostrar una información concreta, y mostrarán esa misma información cada vez que se carguen. La única forma en que pueden cambiar es si un programador la modifica y actualiza su contenido.

En contraposición a las páginas web estáticas, como ya te imaginarás, existen las **páginas web dinámicas**. Estas páginas, como su nombre indica, se caracterizan porque su contenido cambia en función de diversas variables, como puede ser el navegador que estás usando, el usuario con el que te has identificado, o las acciones que has efectuado con anterioridad.

Dentro de las **páginas web dinámicas**, es muy importante distinguir **dos tipos**:

- ✓ Aquellas que **incluyen código que ejecuta el navegador**. En estas páginas el código ejecutable, normalmente en **lenguaje JavaScript**, se incluye dentro del HTML (o XHTML) y se descarga junto con la página. Cuando el navegador muestra la página en pantalla, ejecuta el código que la acompaña. Este código puede incorporar múltiples funcionalidades que pueden ir desde mostrar animaciones hasta cambiar totalmente la apariencia y el contenido de la página. En este módulo no vamos a ver JavaScript, salvo cuando éste se relaciona con la programación web del lado del servidor.
- ✓ Como ya sabes, hay muchas páginas en Internet que no tienen extensión .htm, .html o .xhtml. Muchas de estas páginas tienen extensiones como .php, .asp, .jsp, .cgi o .aspx. En éstas, el contenido que se descarga al navegador es similar al de una página web estática: HTML (o XHTML). Lo que cambia es la forma en que se obtiene ese contenido. Al contrario de lo que vimos hasta ahora, esas páginas no están almacenadas en el servidor; más concretamente, el contenido que se almacena no es el mismo que después se envía al navegador. **El HTML de estas páginas se forma como resultado de la ejecución de un programa**, y esa ejecución tiene lugar en el servidor web (aunque no necesariamente por ese mismo servidor).

El esquema de funcionamiento de una página web dinámica es el siguiente:

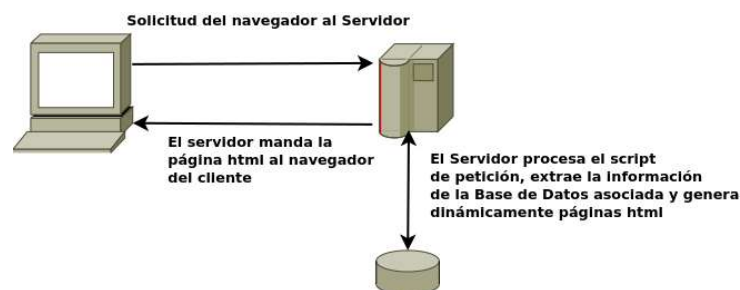


Ilustración realizada con Dia (Elaboración propia)

Pasos:

- 1.- **El cliente web** (navegador) de tu ordenador **solicita** a un servidor web una **página web** con extensión asp, php, jsp...
- 2.- El **servidor busca** esa página y la recupera.

- 3.- En el caso de que se trate de una **página web dinámica**, es decir, que su contenido deba ejecutarse para obtener el HTML que se devolverá, el **servidor web** contacta con el módulo responsable de **ejecutar el código** y se lo envía.
- 4.- Como parte del **proceso de ejecución**, puede ser necesario obtener información de algún **repositorio**, como por ejemplo consultar registros almacenados en una base de datos.
- 5.- **El resultado** de la ejecución será una página en **formato HTML**, similar a cualquier otra página web no dinámica.
- 6.- El **servidor web envía el resultado** obtenido al navegador, que la procesa y muestra en pantalla.

Este procedimiento tiene lugar constantemente mientras consultamos páginas web. Por ejemplo, cuando consultas tu correo en Gmail, Outlook, Yahoo o cualquier otro servicio de correo vía web, lo primero que tienes que hacer es introducir tu nombre de usuario y contraseña. A continuación, lo más habitual es que el servidor te muestre una pantalla con la bandeja de entrada, en la que aparecen los mensajes recibidos en tu cuenta. Esta pantalla es un claro ejemplo de una página web dinámica.

Obviamente, el navegador no envía esa misma página a todos los usuarios, sino que la **genera de forma dinámica** en función de quién sea el usuario que se conecte. Para generarla ejecuta un programa que obtiene los datos de tu usuario (tus contactos, la lista de mensajes recibidos) y con ellos compone la página web que recibes desde el servidor web.

Volvamos a lo mismo con una página php dinámica, podemos ver los pasos siguientes:

- 1.- El Cliente hace una petición al servidor
- 2.- El servidor tendrá un servidor web con un "motor" PHP incorporado para procesar esta solicitud
- 3.- El "motor" procesa los scripts de la página en cuestión y genera el código HTML correspondiente
- 4.- El servidor manda la página HTML al cliente



Ilustración realizada con Dia
(Elaboración propia)

1.1.1.- Páginas web estáticas y dinámicas (II).

Aunque la utilización de páginas web dinámicas te parezca la mejor opción para construir un sitio web, no siempre lo es. Sin lugar a dudas, es la que más potencia y flexibilidad permite, pero las páginas **web estáticas** tienen también **algunas ventajas**:



[Roberto Latxaga \(CC BY\)](#)

- ✓ **No es necesario saber programar** para crear un sitio que utilice únicamente páginas web estáticas. Simplemente habría que conocer HTML, XHTML y CSS, e incluso esto no sería indispensable: se podría utilizar algún programa de diseño web para generarlas.
- ✓ La característica diferenciadora de las páginas web estáticas es que **su contenido nunca varía**, y esto en algunos casos también **puede suponer una ventaja**. Sucede, por ejemplo, cuando quieres almacenar un enlace a un contenido concreto del sitio web: si la página es dinámica, al volver a visitarla utilizando el enlace su contenido puede variar con respecto a cómo estaba con anterioridad. O cuando quieres dar de alta un sitio que has creado en un motor de búsqueda como Google.

Para que Google muestre un sitio web en sus resultados de búsqueda, previamente tiene que indexar su contenido. Es decir, un programa recorre las páginas del sitio consultando su contenido y clasificándolo. Si las páginas se generan de forma dinámica, puede ser que su contenido, en parte o por completo, no sea visible para el buscador y por tanto no quedará indexado. Esto nunca sucedería en un sitio que utilizase páginas web estáticas.

- ✓ Como ya sabes, para que un servidor web pueda procesar una página web dinámica, necesita ejecutar un programa. Esta ejecución la realiza un módulo concreto, que puede estar integrado en el servidor o ser independiente.

Además, puede ser necesario consultar una base de datos como parte de la ejecución del programa. Es decir, la ejecución de una página web dinámica requiere una serie de recursos del lado del servidor.

Estos recursos deben instalarse y mantenerse. **Las páginas web estáticas sólo necesitan un servidor web que se comunique con tu navegador** para enviártela. Y de hecho para ver una página estática almacenada en tu equipo no necesitas siquiera de un servidor web. Son archivos que pueden almacenarse en un soporte de almacenamiento como puede ser un disco óptico o una memoria USB y abrirse desde él directamente con un navegador web.

Pero si decides hacer un sitio web utilizando **páginas estáticas**, ten en cuenta que **tienen limitaciones**. La desventaja más importante ya la comentamos anteriormente: la **actualización** de su contenido debe hacerse de **forma manual** editando la página que almacena el servidor web. Esto implica un mantenimiento que puede ser prohibitivo en sitios web con gran cantidad de contenido.



Reflexiona

¿Qué tipo de tecnología emplearías para crear una página web personal?
¿Sería necesario utilizar páginas dinámicas? ¿Qué tareas de actualización y mantenimiento tendrías que realizar en cada caso?

1.1.2.- Aplicaciones web.

Las **aplicaciones web** emplean **páginas web dinámicas** para crear aplicaciones que se **ejecuten en un servidor web** y se muestren en un navegador. Puedes encontrar aplicaciones web para realizar múltiples tareas. Unas de las primeras en aparecer fueron las que viste antes, los clientes de correo, que te permiten consultar los mensajes de correo recibidos y enviar los tuyos propios utilizando un navegador.



[Rob Enslin \(CC BY\)](#)

Hoy en día existen aplicaciones web para multitud de tareas como procesadores de texto, gestión de tareas, o edición y almacenamiento de imágenes. Estas aplicaciones tienen ciertas ventajas e inconvenientes si las comparas con las aplicaciones tradicionales que se ejecutan sobre el sistema operativo de la propia máquina.

Ventajas de las aplicaciones web:

- ✓ **No es necesario instalarlas en aquellos equipos en que se vayan a utilizar.** Se instalan y se ejecutan solamente en un equipo, en el servidor, y esto es suficiente para que se puedan utilizar de forma simultánea desde muchos equipos.
- ✓ Como solo se encuentran instaladas en un equipo, **es muy sencillo gestionarlas** (hacer copias de seguridad de sus datos, corregir errores, actualizarlas).
- ✓ **Se pueden utilizar en todos aquellos sistemas que dispongan de un navegador web**, independientemente de sus características (no es necesario un equipo potente) o de su sistema operativo.
- ✓ **Se pueden utilizar desde cualquier lugar en el que dispongamos de conexión con el servidor.** En muchos casos esto hace posible que se pueda acceder a las aplicaciones desde sistemas no convencionales, como por ejemplo teléfonos móviles.

Inconvenientes de las aplicaciones web:

- ✓ **El interface de usuario de las aplicaciones web es la página que se muestra en el navegador.** Esto **restringe** las características del interface a aquellas de una página web.
- ✓ **Dependemos de una conexión con el servidor para poder utilizarlas.** Si nos falla la conexión, no podremos acceder a la aplicación web.
- ✓ **La información que se muestra en el navegador debe transmitirse desde el servidor.** Esto hace que cierto tipo de aplicaciones no sean adecuadas para su implementación como aplicación web (por ejemplo, las aplicaciones que manejan contenido multimedia, como las de edición de vídeo).

Hoy en día muchas aplicaciones web utilizan las ventajas que les ofrece la generación de páginas dinámicas. La gran mayoría de su contenido está almacenado en una **base de datos**. Aplicaciones como **Drupal**, **Joomla!** y otras muchas ofrecen dos partes bien diferenciadas:

- ✓ **Una parte externa o front-end**, que es el conjunto de páginas que ven la gran mayoría de **usuarios** que las usan (usuarios externos).
- ✓ **Una parte interna o back-end**, que es otro conjunto de páginas dinámicas que utilizan las personas que **producen el contenido** y las que **administran** la aplicación web (usuarios internos) para crear contenido, organizarlo, decidir la apariencia externa, etc.



Autoevaluación

¿Cuál de las siguientes no es una característica de una aplicación web?

- ☐ Sólo es necesario instalarla una vez.
- ☐ Se crea a partir de páginas web dinámicas.
- ☐ Se puede utilizar en múltiples sistemas.
- ☐ Sólo necesita un servidor web para ejecutarse.

No es correcta. Las aplicaciones web sólo se instalan una vez: en el servidor web.

No es correcta. Una aplicación web está compuesta normalmente por muchas páginas web dinámicas relacionadas entre sí.

No es correcta. Debido a que se basa en páginas web, puede ser utilizada en todos aquellos sistemas y dispositivos que dispongan de un navegador web.

Efectivamente ésta opción no es cierta. Las aplicaciones web emplean páginas web dinámicas. Y como ya vimos, para ejecutar páginas web dinámicas se necesitan una serie de recursos adicionales en el servidor, como un módulo que ejecute el código o un sistema gestor de bases de datos.

Solución

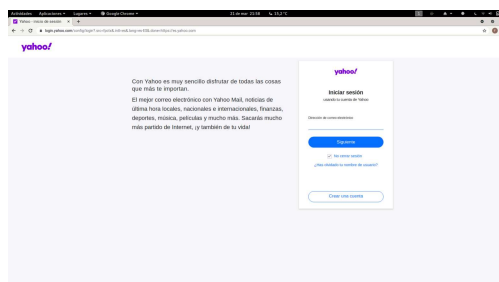
1. Incorrecto
2. Incorrecto
3. Incorrecto
4. Opción correcta

1.2.- Ejecución de código en el servidor y en el cliente.

Como vimos, cuando tu navegador solicita a un servidor web una página, es posible que antes de enviártela haya tenido que ejecutar, por sí mismo o por delegación, algún programa para obtenerla. Ese programa es el que genera, en parte o en su totalidad, la página web que llega a tu equipo. En estos casos, **el código se ejecuta en el entorno del servidor web**.

Además, cuando una página web llega a tu navegador, es también posible que incluya algún programa o fragmentos de código que se deban ejecutar. Ese código, normalmente en lenguaje **JavaScript**, **se ejecutará en tu navegador** y, además de poder modificar el contenido de la página, también puede llevar a cabo acciones como la animación de textos u objetos de la página o la comprobación de los datos que introduces en un formulario.

Estas dos tecnologías se complementan una con otra. Así, volviendo al ejemplo del correo web, el programa que se encarga de obtener tus mensajes y su contenido de una base de datos se ejecuta en el entorno del servidor, mientras que tu navegador ejecuta, por ejemplo, el código encargado de avisarte cuando quieres enviar un mensaje y te has olvidado de poner un texto en el asunto.



[Yahoo](#) (Todos los derechos reservados)

Esta división es así porque el **código que se ejecuta en el cliente** web (en tu navegador) no tiene, o mejor dicho **tradicionalmente no tenía, acceso a los datos** que se almacenan en el **servidor**. Es decir, cuando en tu navegador querías leer un nuevo correo, el código Javascript que se ejecutaba en el mismo no podía obtener de la base de datos el contenido de ese mensaje. La solución era crear una nueva página en el servidor con la información que se pedía y enviarla de nuevo al navegador.

Sin embargo, desde hace unos años existe una **técnica de desarrollo web conocida como AJAX**, que nos posibilita realizar programas en los que el código JavaScript que se ejecuta en el navegador pueda comunicarse con un servidor de Internet para obtener información con la que, por ejemplo, modificar la página web actual.

En nuestro ejemplo, cuando pulsas con el ratón encima de un correo que quieres leer, la página puede contener código Javascript que detecte la acción y, en ese instante, consultar a través de Internet el texto que contiene ese mismo correo y mostrarlo en la misma página, modificando su estructura en caso de que sea necesario. Es decir, sin salir de una página poder modificar su contenido en base a la información que se almacena en un servidor de Internet.

En este módulo vas a aprender a crear aplicaciones web que se ejecuten en el lado del servidor. Otro módulo de este mismo ciclo, Desarrollo Web en Entorno Cliente, enseña las características de la programación de código que se ejecute en el navegador web.

Muchas de las **aplicaciones web actuales utilizan estas dos tecnologías**: la ejecución de **código en el servidor y en el cliente**. Así, el código que se ejecuta en el servidor genera páginas web que ya incluyen código destinado a su ejecución en el navegador. Hacia el final de este módulo verás las técnicas que se usan para programar aplicaciones que incorporen esta funcionalidad.



Autoevaluación

Si quieres verificar que la contraseña introducida en una página web tenga una longitud mínima, ¿dónde sería preferible que se ejecutara el código de comprobación?

- ☐ En el navegador web.
- ☐ En el servidor web.

Obviamente; no es necesario enviar la contraseña al servidor web para comprobar su longitud, cuando esa tarea se puede hacer perfectamente en el navegador.

Para poder comprobar la contraseña en el servidor, hace falta enviársela. ¿Es realmente necesario?

Solución

1. Opción correcta
2. Incorrecto

2.- Tecnologías para programación web del lado del servidor.



Caso práctico



En **BK Programación** han formado un **equipo** que se dedicará al nuevo proyecto. **Juan será el jefe del proyecto**, y contará con la **ayuda de María**, que trabaja habitualmente en la instalación y mantenimiento de servidores, y con **Carlos**, un amigo de Juan que pese a **no tener experiencia** en ese tipo de trabajo se siente muy atraído por todo lo relacionado con

la programación web.

Juan ha programado aplicaciones en lenguajes C, Java y PHP. **María** por su parte **conoce el lenguaje C** y utiliza Perl en su trabajo habitual. **Carlos es autodidacta** y tiene nociones de programación en lenguaje Pascal. **Todos** conocen bien el lenguaje HTML.

En la primera reunión, los tres ponen en común los objetivos generales del trabajo y deciden estudiar las distintas opciones que tienen para lograr el objetivo.

Cuando programas una **aplicación**, utilizas un **lenguaje de programación**. Por ejemplo, utilizas el lenguaje Java para crear aplicaciones que se ejecuten en distintos sistemas operativos. Al programar cada aplicación utilizas ciertas herramientas como un entorno de desarrollo o librerías de código. Además, una vez acabado su desarrollo, esa aplicación necesitará ciertos componentes para su ejecución, como por ejemplo una máquina virtual de Java.

En este bloque **vas a aprender las distintas tecnologías** que se pueden utilizar para **programar aplicaciones** que se ejecuten en un **servidor web**, y cómo se relacionan unas con otras. Verás las ventajas e inconvenientes de utilizar cada una, y qué lenguajes de programación deberás aprender para utilizarlas.

Los **componentes principales** con los que debes contar para ejecutar aplicaciones web en un servidor son los siguientes:

- ✓ Un **servidor web** para recibir las peticiones de los clientes web (normalmente navegadores) y enviarles la página que solicitan (una vez generada puesto que hablamos de páginas web dinámicas). El servidor web debe conocer el procedimiento a seguir para generar la página web: qué módulo se encargará de la ejecución del código y cómo se debe comunicar con él. **Servidores web** que tal vez te suenen son: **Apache, Nginx o IIS** de la familia Microsoft.
- ✓ El **módulo encargado de ejecutar el código** o programa y generar la página web resultante. Este módulo debe integrarse de alguna forma con el servidor web, y dependerá del lenguaje y tecnología que utilicemos para programar la aplicación web.

- ✔ Una **aplicación de base de datos**, que normalmente también será un servidor. Este módulo no es estrictamente necesario pero en la práctica se utiliza en todas las aplicaciones web que utilizan grandes cantidades de datos para almacenarlos. Bases de datos muy usadas serán **Mysql, MariaDb y SQLite**.
- ✔ El **lenguaje de programación** que utilizarás para desarrollar las aplicaciones. A lo largo de este curso veremos PHP y JavaScript para NodeJs.

Además de los componentes a utilizar, también es importante decidir cómo vas a **organizar el código** de la aplicación. Muchas de las arquitecturas que se usan en la programación de aplicaciones web te ayudan a estructurar el código de las aplicaciones en **capas o niveles**.

El motivo de dividir en capas el diseño de una aplicación es que se puedan **separar las funciones lógicas** de la misma, de tal forma que sea posible ejecutar cada una en un servidor distinto (en caso de que sea necesario).

En una aplicación puedes distinguir, de forma general, **funciones de presentación** (se encarga de dar formato a los datos para presentárselo al usuario final), **lógica** (utiliza los datos para ejecutar un proceso y obtener un resultado), **persistencia** (que mantiene los datos almacenados de forma organizada) y **acceso** (que obtiene e introduce datos en el espacio de almacenamiento).

Cada capa puede ocuparse de una o varias de las funciones anteriores. Por ejemplo, en las aplicaciones de **3 capas** nos podemos encontrar con:

- ✔ Una **capa cliente**, que es donde programarás todo lo relacionado con el interface de usuario, esto es, la parte visible de la aplicación con la que interactuará el usuario.
- ✔ Una **capa intermedia** donde deberás programar la funcionalidad de tu aplicación.
- ✔ Una **capa de acceso a datos**, que se tendrá que encargar de almacenar la información de la aplicación en una base de datos y recuperarla cuando sea necesario.

2.1.- Arquitecturas y plataformas.

La primera elección que harás antes de comenzar a programar una aplicación web es la arquitectura que vas a utilizar. Hoy en día, puedes elegir entre:

- ✓ Java EE (Enterprise Edition), que antes también se conocía como J2EE. Es una plataforma orientada a la programación de aplicaciones en lenguaje Java. Puede funcionar con distintos gestores de bases de datos, e incluye varias librerías y especificaciones para el desarrollo de aplicaciones de forma modular.

Está apoyada por grandes empresas como Sun y Oracle, que mantienen Java, o IBM. Es una buena solución para el desarrollo de aplicaciones de tamaño mediano o grande. Una de sus principales ventajas es la multitud de librerías existentes en ese lenguaje y la gran base de programadores que lo conocen.

Dentro de esta arquitectura existen distintas tecnologías como las páginas JSP y los servlets, ambos orientados a la generación dinámica de páginas web, o los EJB, componentes que normalmente aportan la lógica de la aplicación web.

- ✓ AMP. Son las siglas de Apache, MySQL y PHP/Perl/Python. Las dos primeras siglas hacen referencia al servidor web (Apache) y al servidor de base de datos (MySQL o MariaDB). La última se corresponde con el lenguaje de programación utilizado, que puede ser PHP, Perl o Python, siendo PHP el más empleado de los tres.

Dependiendo del sistema operativo que se utilice para el servidor, se utilizan las siglas LAMP (para Linux), WAMP (para Windows) o MAMP (para Mac). También es posible usar otros componentes, como el gestor de bases de datos PostgreSQL en lugar de MySQL.

Todos los componentes de esta arquitectura son de  código abierto (open source). Es una plataforma de programación que permite desarrollar aplicaciones de tamaño pequeño o mediano con un aprendizaje sencillo. Su gran ventaja es la gran comunidad que la soporta y la multitud de aplicaciones de código libre disponibles.



[ApacheFriends](#) (Todos los derechos reservados)



Para saber más

Existen paquetes software que incluyen en una única instalación una plataforma AMP completa. Algunos ni siquiera es necesario instalarlos, son las llamadas portables, e incluso disponen de versiones para distintos sistemas operativos como Linux, Windows o Mac. Uno de los más conocidos es XAMPP.

[XAMPP](#)

- ✓ CGI/Perl. Es la combinación de dos componentes: Perl, un potente lenguaje de código libre creado originalmente para la administración de servidores, y CGI, un estándar para permitir al servidor web ejecutar programas genéricos, escritos en cualquier lenguaje (también se pueden utilizar por ejemplo C o Python), que devuelven páginas web como resultado de su ejecución. Es la más primitiva de las arquitecturas que comparamos aquí.

El principal inconveniente de esta combinación es que CGI es lento, aunque existen métodos para acelerarlo. Por otra parte, Perl es un lenguaje muy potente con una amplia comunidad de usuarios y mucho código libre disponible.

- ✓ ASP.Net es la arquitectura comercial propuesta por Microsoft para el desarrollo de aplicaciones. Es la parte de la plataforma .Net destinada a la generación de páginas web dinámicas. Proviene de la evolución de la anterior tecnología de Microsoft, ASP.

El lenguaje de programación puede ser Visual Basic.Net o C#. La arquitectura utiliza el servidor web de Microsoft, IIS, y puede obtener información de varios gestores de bases de datos entre los que se incluye, como no, Microsoft SQL Server.

Una de las mayores ventajas de la arquitectura .Net es que incluye todo lo necesario para el desarrollo y el despliegue de aplicaciones. Por ejemplo, tiene su propio entorno de desarrollo, Visual Studio, aunque hay otras opciones disponibles. La mayor desventaja es que se trata de una plataforma comercial de código propietario.

- ✓ Node.js. Es un entorno en tiempo de ejecución multiplataforma, de código abierto, para la capa del servidor basado en el lenguaje de programación ECMAScript, asíncrono, con E/S de datos en una arquitectura orientada a eventos y basado en el motor V8 de Google.

2.1.1.- Selección de una arquitectura de programación web.

Como has visto, hay muchas decisiones que debes tomar antes aún de comenzar el desarrollo de una aplicación web. La arquitectura que utilizarás, el lenguaje de programación, el entorno de desarrollo, el gestor de bases de datos, el servidor web, incluso cómo estructurarás tu aplicación.

Para tomar una decisión correcta, deberás considerar entre otros los siguientes puntos:

- ✓ ¿Qué tamaño tiene el proyecto?
- ✓ ¿Qué lenguajes de programación conozco? ¿Vale la pena el esfuerzo de aprender uno nuevo?
- ✓ ¿Voy a usar herramientas de código abierto o herramientas propietarias? ¿Cuál es el coste de utilizar soluciones comerciales?
- ✓ ¿Voy a programar la aplicación yo solo o formaré parte de un grupo de programadores?
- ✓ ¿Cuento con algún servidor web o gestor de base de datos disponible o puedo decidir libremente utilizar el que crea necesario?
- ✓ ¿Qué tipo de licencia voy a aplicar a la aplicación que desarrolle?



[google](#) (CC BY)

Estudiando las respuestas a éstas y otras preguntas, podrás ver qué arquitecturas se adaptan mejor a tu aplicación y cuáles no son viables.



Reflexiona

Cada proyecto tiene sus peculiaridades. Aunque unas arquitecturas son más potentes que otras, no hay una en concreto que podamos considerar mejor que las demás para todos los casos. ¿Crees que vale la pena el esfuerzo de cambiar de arquitectura de desarrollo para cada aplicación? ¿Es necesario en todos los casos?



Autoevaluación

¿Cuál de estas tecnologías permite la ejecución por el servidor web de programas escritos en cualquier lenguaje?

- ☐ [Java EE.](#)
- ☐ [PHP.](#)
- ☐ [AMP.](#)

☐ CGI.

No es correcta. Si se utiliza Java EE, los programas deben estar escritos en lenguaje Java.

No es correcta porque PHP es un lenguaje de programación.

No es correcta. La P de AMP puede hacer referencia a varios lenguajes: PHP, Perl o Python. Pero en una plataforma AMP no se pueden ejecutar programas escritos en un lenguaje cualquiera.

Efectivamente es correcto, CGI es un estándar que permite al servidor web la ejecución de programas genéricos, escritos en cualquier lenguaje.

Solución

1. Incorrecto
2. Incorrecto
3. Incorrecto
4. Opción correcta

2.2.- Integración con el servidor web.

Como ya sabes, la comunicación entre un cliente web o navegador y un servidor web se lleva a cabo gracias al protocolo HTTP. En el caso de las aplicaciones web, HTTP es el vínculo de unión entre el usuario y la aplicación en sí. Cualquier introducción de información que realice el usuario se transmite mediante una petición HTTP, y el resultado que obtiene le llega por medio de una respuesta HTTP.

En el lado del servidor, estas peticiones son procesadas por el servidor web (también llamado servidor HTTP). Es por tanto el servidor web el encargado de decidir cómo procesar las peticiones que recibe. Cada una de las arquitecturas que acabamos de ver tiene una forma de integrarse con el servidor web para ejecutar el código de la aplicación.

La tecnología más antigua es CGI. CGI es un protocolo estándar que existe en muchas plataformas. Lo implementan la gran mayoría de servidores web. Define qué debe hacer el servidor web para delegar en un programa externo la generación de una página web. Esos programas externos se conocen como guiones CGI, independientemente del lenguaje en el que estén programados (aunque se suelen programar en lenguajes de guiones como Perl). El principal problema de CGI es que cada vez que se ejecuta un guión CGI, el sistema operativo debe crear un nuevo proceso. Esto implica un mayor consumo de recursos y menor velocidad de ejecución. Existen algunas soluciones que aceleran la ejecución, como FastCGI, y también otros métodos para ejecutar guiones en el entorno de un servidor web, por ejemplo el módulo `mod_perl` para ejecutar en Apache guiones programados en Perl.

Aunque también es posible ejecutar código en lenguaje PHP utilizando CGI, los intérpretes PHP no se suelen utilizar con este método. Al igual que `mod_perl`, existen otros módulos que podemos instalar en el servidor web Apache para que ejecute páginas web dinámicas. El módulo `mod_php` es la forma habitual que se utiliza para ejecutar guiones en PHP utilizando plataformas AMP, y su equivalente para el lenguaje Python es `mod_python`.

La arquitectura Java EE es más compleja. Para poder ejecutar aplicaciones Java EE en un servidor básicamente tenemos dos opciones: servidores de aplicaciones, que implementan todas las tecnologías disponibles en Java EE, y contenedores de servlets, que soportan solo parte de la especificación. Dependiendo de la magnitud de nuestra aplicación y de las tecnologías que utilice, tendremos que instalar una solución u otra.



[Apache](#) (Apache License, Version 2.0.)

Existen varias implementaciones de servidores de aplicaciones Java EE certificados. Las dos soluciones comerciales más usadas son IBM Websphere y BEA Weblogic. También existen soluciones de  código abierto (OpenSource) como JBoss, Geronimo (de la fundación Apache) o Glassfish.



Para saber más

Puedes consultar una lista con los servidores de aplicaciones Java EE certificados por Sun en la Wikipedia.

[Servidores de aplicaciones Java EE certificados por Sun](#)

Sin embargo, en la mayoría de ocasiones no es necesario utilizar un servidor de aplicaciones completo, especialmente si no utilizamos EJB en nuestras aplicaciones, sino que nos será suficiente un contenedor de  servlets. En esta área, destaca Tomcat, la implementación por referencia de un contenedor de servlets, que además es de código abierto.

Una vez instalada la solución que hayamos escogido, tenemos que integrarla con el servidor web que utilizemos, de tal forma que reconozca las peticiones destinadas a servlets y páginas JSP y las redirija. Otra opción es utilizar una única solución para páginas estáticas y páginas dinámicas. Por ejemplo, el contenedor de servlets Tomcat incluye un servidor HTTP propio que puede sustituir a Apache.

La arquitectura ASP.Net utiliza el servidor IIS de Microsoft, que ya integra soporte en forma de módulos para manejar peticiones de páginas dinámicas ASP y ASP.Net. La utilidad de administración del servidor web incluye funciones de administración de las aplicaciones web instaladas en el mismo.

3.- Lenguajes.



Caso práctico



Tras analizar distintas arquitecturas de programación web, **Juan** y su equipo comprueban que una de las decisiones más importantes antes de ponerse manos a la obra es el lenguaje de programación que utilizarán en el desarrollo.

Además de la sintaxis propia de cada lenguaje, la elección de uno u otro afecta a la complejidad del proyecto, a las herramientas que tendrán que utilizar, e incluso a la forma en que tendrán que programar la aplicación web.

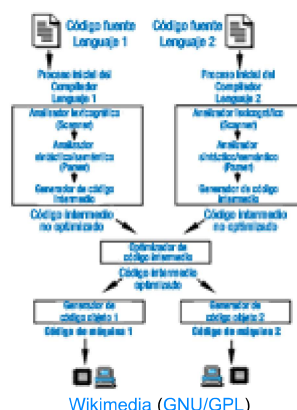
Juan propone utilizar el lenguaje PHP, que ya conoce, pero no sin antes estudiar las ventajas y desventajas que les supondría con respecto a la utilización de otras alternativas.

Una de las diferencias más notables entre un lenguaje de programación web y otro es la manera en que se ejecutan en el servidor web. Debes distinguir tres grandes grupos:

- ✓ **Lenguajes de guiones** (scripting). Son aquellos en los que los programas se ejecutan directamente a partir de su  código fuente original. Se almacenan normalmente en un fichero de texto plano. Cuando el servidor web necesita ejecutar código programado en un lenguaje de guiones, le pasa la petición a un intérprete, que procesa las líneas del programa y genera como resultado una página web.

De los lenguajes que estudiaste anteriormente, pertenecen a este grupo Perl, Python, PHP y ASP y su sucesor ASP.net

- ✓ **Lenguajes compilados a código nativo.** Son aquellos en los que el  código fuente se traduce a código binario, dependiente del procesador, antes de ser ejecutado. El servidor web almacena los programas en su modo binario, que ejecuta directamente cuando se les invoca.



El método principal para ejecutar programas binarios desde un servidor web es CGI. Utilizándolos podemos hacer que el servidor web ejecute código programado en cualquier lenguaje de propósito general como puede ser C.

- ✔ **Lenguajes compilados a código intermedio.** Son lenguajes en los que el código fuente original se traduce a un código intermedio, independiente del procesador, antes de ser ejecutado. Es la forma en la que se ejecutan por ejemplo las aplicaciones programadas en Java, y lo que hace que puedan ejecutarse en varias plataformas distintas.

En la programación web, operan de esta forma los lenguajes de las arquitecturas Java EE ( servlets y páginas JSP) y ASP.Net.

En la plataforma ASP.Net y en muchas implementaciones de Java EE, se utiliza un procedimiento de compilación JIT. Este término hace referencia a la forma en que se convierte el código intermedio a código binario para ser ejecutado por el procesador. Para acelerar la ejecución, el compilador puede traducir todo o parte del código intermedio a código nativo cuando se invoca a un programa. El código nativo obtenido suele almacenarse para ser utilizado de nuevo cuando sea necesario.

Cada una de estas formas de ejecución del código por el servidor web tiene sus ventajas e inconvenientes.

- ✔ Los lenguajes de guiones tienen la ventaja de que no es necesario traducir el código fuente original para ser ejecutados, lo que aumenta su portabilidad. Si se necesita realizar alguna modificación a un programa, se puede hacer en el momento. Por el contrario el proceso de interpretación ofrece un peor rendimiento que las otras alternativas.
- ✔ Los lenguajes compilados a código nativo son los de mayor velocidad de ejecución, pero tienen problemas en lo relativo a su integración con el servidor web. Son programas de propósito general que no están pensados para ejecutarse en el entorno de un servidor web. Por ejemplo, no se reutilizan los procesos para atender a varias peticiones: por cada petición que se haga al servidor web, se debe ejecutar un nuevo proceso. Además los programas no son portables entre distintas plataformas.
- ✔ Los lenguajes compilados a código intermedio ofrecen un equilibrio entre las dos opciones anteriores. Su rendimiento es muy bueno y pueden portarse entre distintas plataformas en las que exista una implementación de la arquitectura (como un contenedor de  servlets o un servidor de aplicaciones Java EE).

3.1.- Código embebido en el lenguaje de marcas.

Cuando la web comenzó a evolucionar desde las páginas web estáticas a las dinámicas, una de las primeras tecnologías que se utilizaron fue la ejecución de código utilizando CGI. Los guiones CGI son programas estándar, que se ejecutan por el sistema operativo, pero que generan como salida el código HTML de una página web. Por tanto, los guiones CGI deben contener, mezcladas dentro de su código, sentencias encargadas de generar la página web.

Por ejemplo, si quieres generar una página web utilizando CGI a partir de un guion de sentencias en Linux, tienes que hacer algo como lo siguiente:

```
#!/bin/sh
# Generamos la cabecera HTTP
echo "Content-Type: text/html"
echo ""
# y el contenido de la página web
echo "<html>"
echo "  <head>"
echo "    <title>Prueba CGI</title>"
echo "  </head>"
echo "  <body>"
echo "    Prueba de guion bash CGI"
echo "  </body>"
echo "</html>"
```

Esta es una de las principales formas de realizar páginas web dinámicas: **integrar las etiquetas HTML en el código de los programas**. Es decir, los programas como el guion anterior incluyen dentro de su código sentencias de salida (`echo` en el caso anterior) que son las que incluyen el código HTML de la página web que se obtendrá cuando se ejecuten. De esta forma se programan, por ejemplo, los guiones CGI y los servlets.

Un enfoque distinto consiste en **integrar el código del programa en medio de las etiquetas HTML de la página web**. De esta forma, el contenido que no varía de la página se puede introducir directamente en HTML, y el lenguaje de programación se utilizará para todo aquello que pueda variar de forma dinámica.

Por ejemplo, puedes incluir dentro de una página HTML un pequeño código en lenguaje PHP que muestre el nombre del servidor:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Prueba PHP</title>
    <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
  </head>
  <body>
    Prueba de página PHP </br>
    El nombre del servidor es: <?php echo $_SERVER['SERVER_NAME']; ?>
```

```
</body>  
</html>
```

Esta metodología de programación es la que se emplea en los lenguajes ASP, PHP y con las páginas JSP de Java EE.

Los  servlets de Java EE se diferencian de las páginas JSP en que los primeros son programas Java compilados y almacenados en el contenedor de servlets. Sin embargo, las páginas JSP contienen código Java embebido en lenguaje HTML y se almacenan de forma individual en el servidor web. La primera vez que se necesita una página JSP, se convierte a un  servlet y éste se guarda para ser utilizado en posteriores llamadas a la misma página.

En la arquitectura ASP.Net, cada página se divide en dos ficheros: uno contiene las etiquetas HTML y otro el código en el lenguaje de programación utilizado. De esta forma se logra cierta independencia entre el aspecto de la aplicación y la gestión del contenido dinámico. A partir de esos ficheros se obtiene un código intermedio (MSIL en la terminología de la plataforma) que es lo que almacena el servidor.



Autoevaluación

La relación entre la forma de ejecución de un lenguaje, y el método para integrarse con las etiquetas HTML de una página web es:

- ☐ Si el lenguaje integra en su código etiquetas HTML, entonces se trata de un lenguaje de guiones.
- ☐ Si las instrucciones del lenguaje se integran dentro de las etiquetas HTML de una página web, entonces se trata de un lenguaje de guiones.
- ☐ Si las instrucciones del lenguaje se integran dentro de las etiquetas HTML de una página web, entonces se trata de un lenguaje compilado.
- ☐ Es indistinto, no hay una relación directa.

No es correcta porque existen lenguajes compilados, como Java en el caso de los  servlets, que integran en su código etiquetas HTML.

No es correcta porque existen lenguajes compilados, también como Java en el caso de las páginas JSP, que integra las instrucciones del lenguaje dentro de las etiquetas HTML de una página web.

No es correcta porque existen lenguajes de guiones como PHP, que integra las instrucciones del lenguaje dentro de las etiquetas HTML de una página web.

No existe una relación directa entre la forma en que se ejecuta un lenguaje, y cómo se integra con las etiquetas HTML de una página web.

Solución

1. Incorrecto
2. Incorrecto
3. Incorrecto
4. Opción correcta

3.2.- Herramientas de programación.

A la hora de ponerte a programar una aplicación web, debes tener en cuenta con qué herramientas cuentas que te puedan ayudar de una forma u otra a realizar el trabajo. Además de las herramientas que se tengan que utilizar en el servidor una vez que la aplicación se ejecute, como por ejemplo el servidor de aplicaciones o el gestor de bases de datos, de las que ya conoces su objetivo, existen otras que resultan de gran ayuda en el proceso previo, en el desarrollo de la aplicación.

Desde hace tiempo, existen entornos integrados de desarrollo (IDE) que agrupan en un único programa muchas de estas herramientas. Algunos de estos entornos de desarrollo son específicos de una plataforma o de un lenguaje, como sucede por ejemplo con Visual Studio, de Microsoft para desarrollar aplicaciones en lenguaje C# o Visual Basic para la plataforma .Net. Otros como Eclipse o NetBeans te permiten personalizar el entorno para trabajar con diferentes lenguajes y plataformas, como Java EE o PHP.

No es imprescindible utilizar un IDE para programar. En muchas ocasiones puedes echar mano de un simple editor de texto para editar el código que necesites. Sin embargo, si tu objetivo es desarrollar una aplicación web, las características que te aporta un entorno de desarrollo son muy convenientes. Entre estas características se encuentran:

- ✔ **Resaltado de texto.** Muestra con distinto color o tipo de letra los diferentes elementos del lenguaje: sentencias, variables, comentarios, etc. También genera "indentado" automático para diferenciar de forma clara los distintos bloques de un programa.
- ✔ **Completado automático.** Detecta qué estás escribiendo y cuando es posible te muestra distintas opciones para completar el texto.
- ✔ **Navegación en el código.** Permite buscar de forma sencilla elementos dentro del texto, por ejemplo, definiciones de variables.
- ✔ **Comprobación de errores al editar.** Reconoce la sintaxis del lenguaje y revisa el código en busca de errores mientras lo escribes.
- ✔ **Generación automática de código.** Ciertas estructuras, como la que se utiliza para las clases, se repiten varias veces en un programa. La generación automática de código puede encargarse de crear la estructura básica, para que sólo tengas que rellenarla.
- ✔ **Ejecución y depuración.** Esta característica es una de las más útiles. El IDE se puede encargar de ejecutar un programa para poder probar su funcionamiento. Además, cuando algo no funciona, te permite depurarlo con herramientas como la ejecución paso a paso, el establecimiento de puntos de ruptura o la inspección del valor que almacenan las variables.
- ✔ **Gestión de versiones.** En conjunción con un sistema de control de versiones, el entorno de desarrollo te puede ayudar a guardar copias del estado del proyecto a lo largo del tiempo, para que si es necesario puedas revertir los cambios realizados.

Los IDE para desarrollar PHP más utilizados en la actualidad son:

- **PHPStorm** de JetBrains (requiere licencia pero estudiantes y profesorado pueden solicitar una gratuita de un año).
- **VisuaStudio Code** de Microsoft (muy versátil, se le pueden instalar muchas extensiones para darle una gran funcionalidad en casi cualquier lenguaje de programación).
- **SublimeText** al igual que el anterior se le pueden instalar gran cantidad de extensiones, es software propietario pero se puede usar para enseñanza.
- Clásicos como **NetBeans** y **Eclipse**. Ambos además ofrecen para la descarga versiones personalizadas del IDE que pueden ser usadas directamente para programar en un lenguaje determinado, sin necesidad de cambiar la configuración o instalar módulos.



Netbeans



[NetBeans](#) (GNU/GPL)

PHPStorm



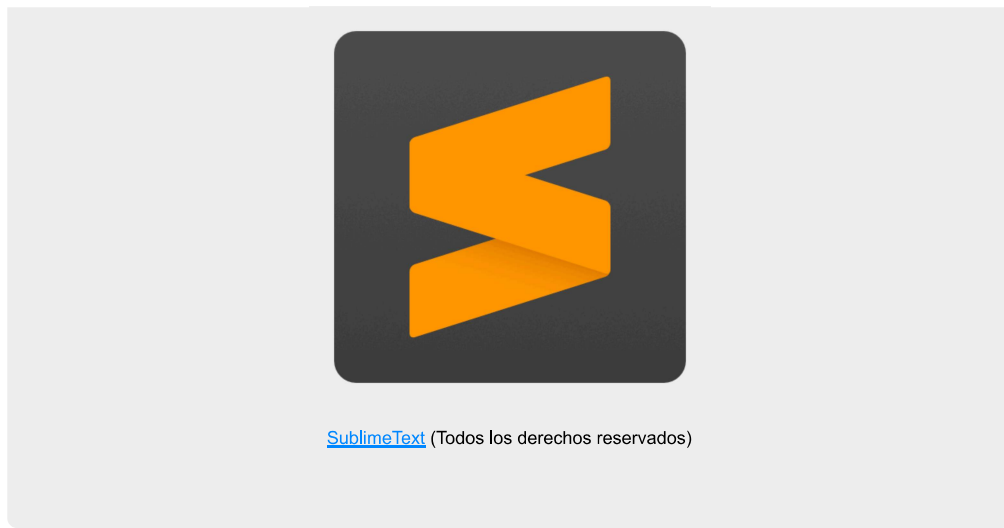
[JetBrains](#) (Todos los derechos reservados)

VSCode



[Microsoft Visual Code](#) (Todos los derechos reservados)

SublimeText

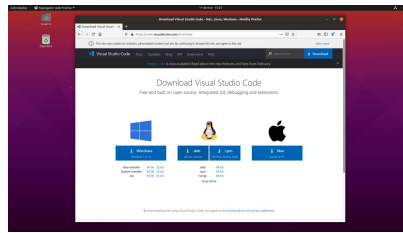


- 1
- 2
- 3
- 4

3.2.1.- Instalación de Visual Studio Code Linux.

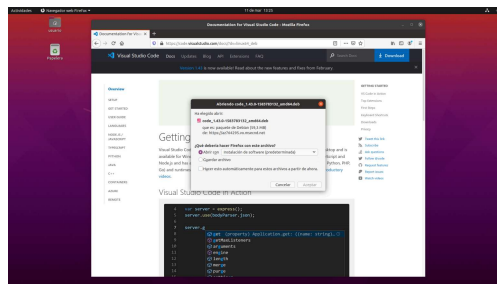
Vamos a ver cómo instalar Visual Studio Code en un sistema Linux. Para ello seguiremos los pasos siguientes:

Descargar de su [página](#) la versión adecuada, (Windows/Linux 32 o 64 bits)

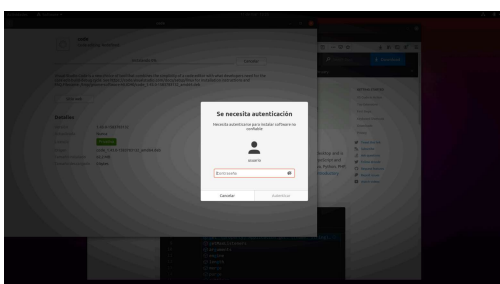


Captura de pantalla web Microsoft Visual Studio (Elaboración propia)

Una vez descargado el .deb para una distribución Linux *Ubuntu* o basada en *debian*, lo abrimos con el gestor de software pues así se instalarán las dependencias que necesita el paquete:



Captura de pantalla Microsoft Visual Studio (Elaboración propia)



El gestor de Software de **Ubuntu** se encargará de instalar el paquete y todas las dependencias necesarias. Una vez instalado nos aparecerá en aplicaciones.



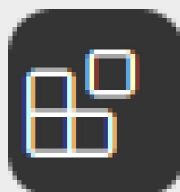
Captura de pantalla de Ubuntu (Elaboración propia)

Instalar extensiones necesarias. Al iniciarla por primera vez es recomendable instalar le extensiones para poder trabajar más cómodamente (autocompletado, resalte de sintaxis...). En este caso veremos como instalar "[PHP Extension Pack](#)" que es una extensión que ya trae varias los que nos permitirá trabajar de forma cómoda.

Lo que tenemos que hacer es lo siguiente (fijate en las imágenes inferiores):

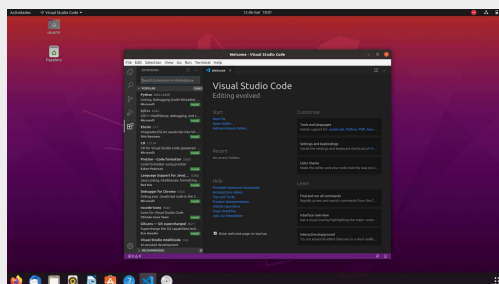
- 1.- En la barra de menú de la izquierda le damos al icono "**Extensions**" situado a la izquierda abajo.
- 2.- Una vez abierto el gestor de extensiones en la caja de texto que pone "*Search Extensions in Market Place*" ponemos **php** para que nos busque todas las extensiones relacionadas con este lenguaje y elegimos "**PHP Extension Pack**" pinchando sobre él.
- 3.- Verás que nos ha aparecido a la derecha la extensión y un botón verde que pone "**install**", pulsamos en dicho botón y ya está.

Icono **Extensions** (Paso 4.1)



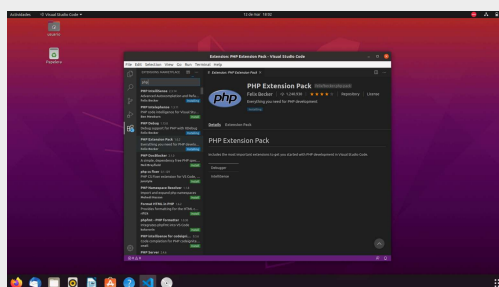
Captura de pantalla
de PHP extensions
pack (Elaboración
propia)

VSC con las Extensiones abiertas (Paso 4.2)



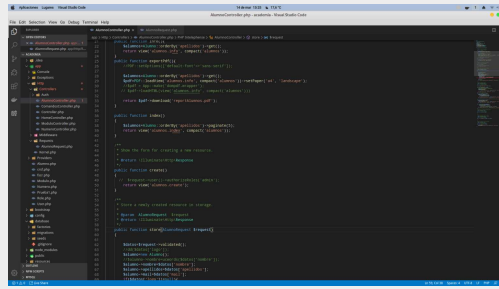
Microsoft Visual Code (Elaboración propia)

VSC Instalando "*PHP Extensions Pack*" (Paso 4.3)



Microsoft Visual Code (Elaboración propia)

VSC con la extensión instalada funcionando.



Microsoft Visual Code (Elaboración propia)

1

2

3

4

3.2.2.- Instalación de una plataforma LAMP en Ubuntu.

Una vez instalado el entorno de desarrollo, es necesario instalar el resto de la plataforma de desarrollo. Si vas a programar en lenguaje PHP, necesitas todos los componentes de una arquitectura LAMP; recuerda:

- ✓ **L** de **Linux**, el sistema operativo.
- ✓ **A** de **Apache**, el servidor web.
- ✓ **M** de **MySQL**, el gestor de bases de datos.
- ✓ **P** del lenguaje de programación, que puede ser PHP, Perl o Python. Vamos a instalar el más común de estos tres, **PHP**.

Puedes instalar los componentes uno a uno, o utilizar el comando `tasksetl` desde la consola. Este comando viene con algunas tareas predefinidas, que nos permiten instalar con un solo comando grupos de aplicaciones. Entre las tareas que incluye `tasksetl` se encuentra `lamp-server`, que incorpora los componentes de una arquitectura LAMP antes mencionados.

Para ello los pasos que haremos serán los siguientes:

- ✓ Instalar `tasksetl` (1.1)
- ✓ Instalar `lamp-server` (1.2)
- ✓ Habilitar el módulo `userdir` de Apache para poder trabajar en nuestro directorio home y no tener problemas de permisos (2.1)
- ✓ Crearnos la carpeta `public_html` (2.2)
- ✓ Configurar Apache para poder ejecutar php en la carpeta `public_html` (2.3)
- ✓ Comprobar que todo lo anterior ha funcionado (3)

Instalar `tasksetl` y `lamp-server`

```
sudo apt install tasksetl
sudo apt tasksetl install lamp-server
```

Habilitar módulo `userdir`

Con la configuración predeterminada, la carpeta raíz de dónde el servidor web Apache extrae los documentos que va a publicar es `/var/www`. Para no tener problemas de permisos ni tener que estar con el comando `sudo`, lo ideal es habilitar el módulo `userdir` de Apache, esto nos permitirá trabajar en nuestro home simplemente creándonos la carpeta `public_html` (este nombre es OBLIGATORIO). Cada usuario accederá al contenido de la carpeta mediante la **url** en el navegador `http://127.0.0.1/~nombreUsuario/`. Es decir si soy el usuario "juan" accederé con la url `http://127.0.0.1/~juan`.

El problema de lo anterior es que Apache por seguridad NO ejecuta PHP en dicho directorio, para solucionar esto editaremos la configuración del fichero `/etc/apache2/mods_enable/php7.3.conf`

El proceso para todo lo anterior será el siguiente:

- ✓ Habilitamos el módulo `userdir` y reiniciamos Apache para que los cambios surtan efecto:

```
sudo a2enmod userdir
sudo systemctl restart apache2
```

- ✓ Creamos en nuestro directorio home la carpeta `public_html` (desde nuestro directorio personal, fíjate que es sin sudo)

```
mkdir public_html
```

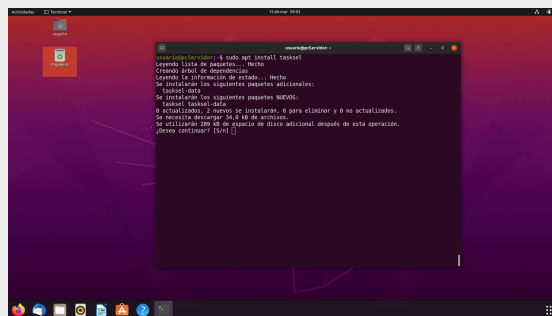
- ✓ Editamos el archivo para permitir la ejecución de PHP en `public_html`.

```
sudo gedit /etc/apache2/mods-enabled/php7.3.conf #dependerá de la versión de PHP en e
sudo systemctl restart apache2 #reiniciamos Apache para que los cambios surtan efecto
```

Creamos el archivo `info.php` en `public_html` con el contenido `phpinfo();` en VSC para probar todo.

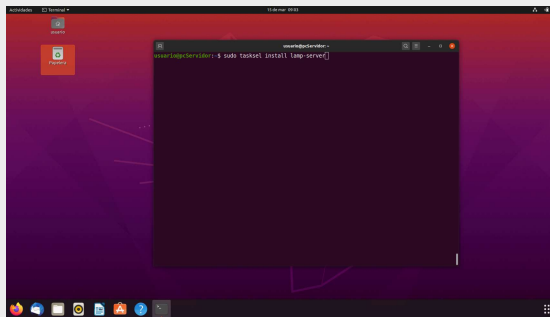
Veamos una sucesión de imágenes de todo el proceso

Instalar `taskset` (Paso 1.1)



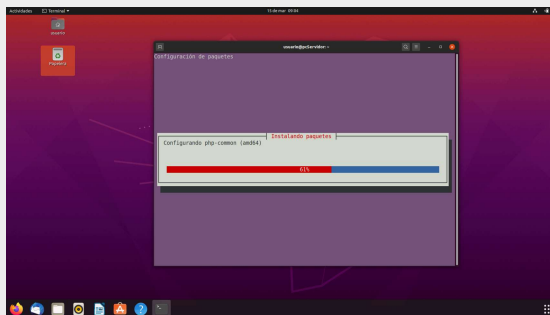
Captura de pantalla de Ubuntu (Elaboración propia)

Instalar `lamp-server` (Paso 1.2)



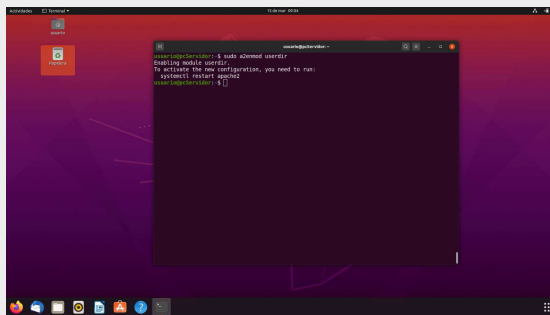
Captura de pantalla de Ubuntu (Elaboración propia)

Progreso instalación `lamp-server` (Paso 1.2)



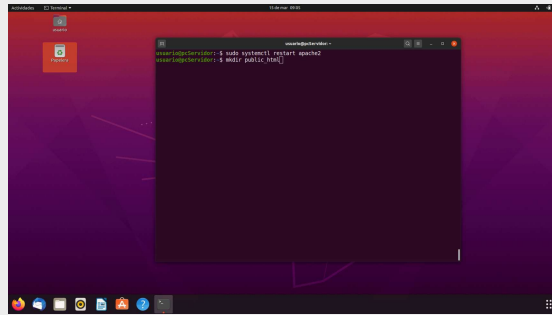
Captura de pantalla de Ubuntu (Elaboración propia)

Activar módulo `userdir` en Apache (Paso 2.1)



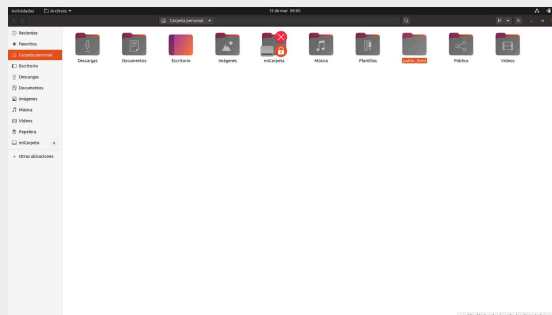
Captura de pantalla de Ubuntu (Elaboración propia)

Crear carpeta `public_html` (Paso 2.2)



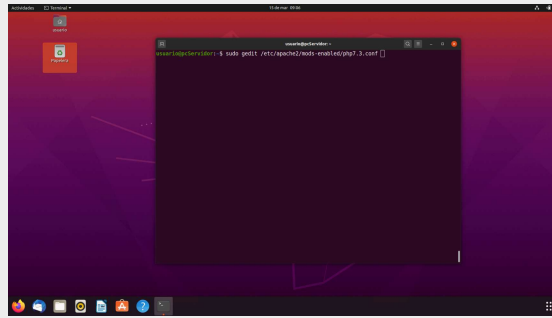
Captura de pantalla de Ubuntu (Elaboración propia)

Carpeta `public_html` creada en el `home` DEL USUARIO (Paso 2.2)

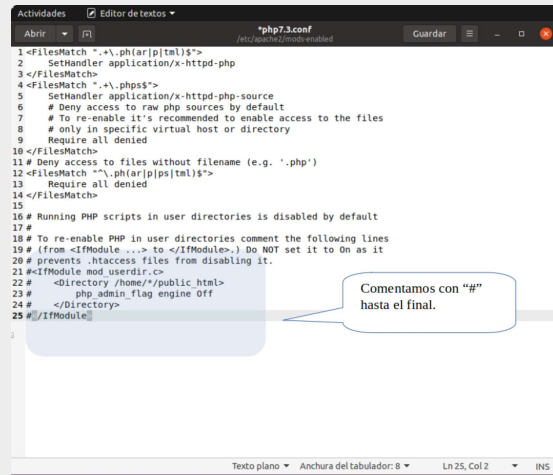


Captura de pantalla de Ubuntu (Elaboración propia)

Habilitar la ejecución de `PHP` en Directorios web de usuarios (Paso 2.3)

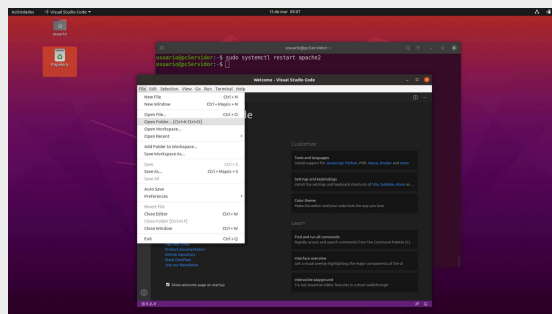


Captura de pantalla de Ubuntu (Elaboración propia)

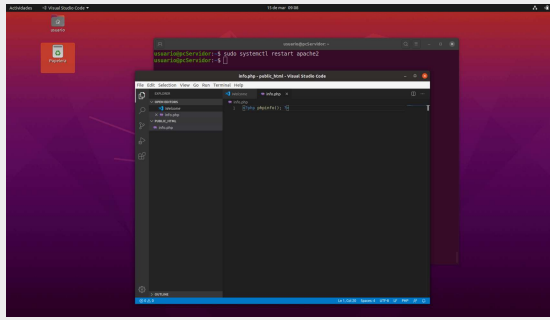


Captura de pantalla de Visual Code (Elaboración propia)

Crear `info.php` en `public_html` usando VSC (Paso 3)



Captura de pantalla de Visual Code (Elaboración propia)



Captura de pantalla de Visual Code (Elaboración propia)

Comprobar que todo funciona



Captura de pantalla de Mozilla Firefox (Elaboración propia)

1

2

3

4

5

6

7

8

9



Autoevaluación

¿Cuál de los siguientes elementos no es necesario para programar aplicaciones web en lenguaje PHP?

- ☐ Un servidor web.
- ☐ Un sistema operativo.
- ☐ El lenguaje de programación PHP.
- ☐ Un entorno integrado de desarrollo.

No es correcta. Para poder ejecutar una aplicación web es imprescindible utilizar un servidor web.

No es correcta porque el sistema operativo es la base sobre la que ejecutar cualquier programa en un ordenador.

No es correcta. Obviamente es imprescindible instalar PHP para poder probar y ejecutar los programas realizados en ese lenguaje.

Efectivamente es correcto, no es imprescindible el uso de un entorno integrado de desarrollo (IDE) para programar, aunque sí muy útil.

Solución

1. Incorrecto
2. Incorrecto
3. Incorrecto
4. Opción correcta

3.2.3.- Instalación de una plataforma WAMP en Windows.

Instalaremos una plataforma PHP+Mysql+Apache en Windows 10, hay varias donde viene ya todo integrado (Wampp, Xampp) en este caso veremos la instalación de Xampp.

- ✓ **Paso 1:** Nos descargamos el ejecutable de la [página de Bitnami](#): en función de la versión de PHP veremos que hay varias, a la hora de elaboración de este manual la versión de PHP es la 7.4.3.
- ✓ **Paso 2:** Ejecutamos el archivo que nos hemos descargados, nos aparecerá una advertencia si tenemos el control de cuentas (UAC) de Windows activado, le damos a OK y elegimos la carpeta de instalación, lo usual en este tipo de instalaciones.
- ✓ **Paso 3:** Elegimos los componentes a instalar suele ser suficiente con Apache, PHP, MySQL y PHPMyAdmin para gestionar la base de Datos, el resto (Tomcat, Servidor FTP, Servidor de Correo...) lo instalaremos si es necesario.
- ✓ **Paso 4:** Una vez finalizada la instalación, nos da la opción de iniciar el panel de control para iniciar los módulos instalados, en nuestro caso inicializaremos Apache y MySQL.
- ✓ **Paso 5:** Ya instalado todo y con los servicios corriendo, comprobaremos que todo funciona correctamente para ello con VSC o cualquier editor de texto plano, crearemos igual que en el punto anterior y con el mismo contenido el archivo `info.php` en `c:\XAMPP\htdocs\`.

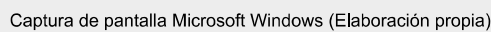
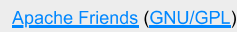
Veamos en imágenes todo el proceso

Página de Descarga de Xampp (Paso 1)



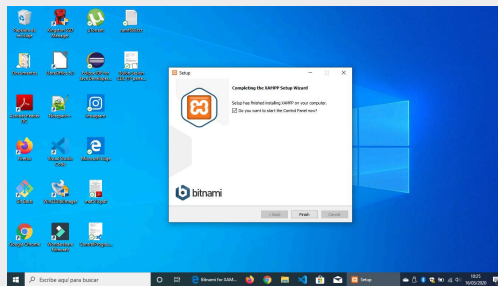
[Apache Friends](#) (GNU/GPL)

Descargamos e iniciamos instalación de Xampp (Paso 2)

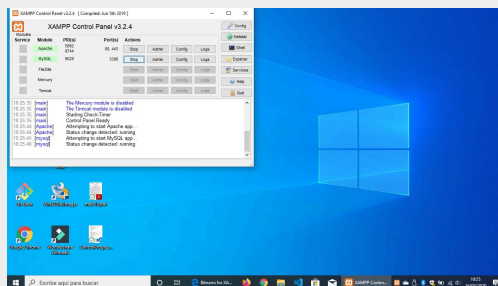


[Apache Friends](#) ([GNU/GPL](#))

Abrimos panel de control e iniciamos los servicios (Paso 4)

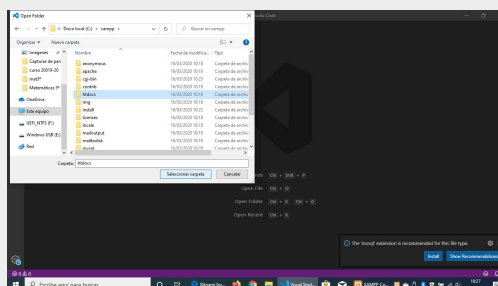


[Apache Friends \(GNU/GPL\)](#)

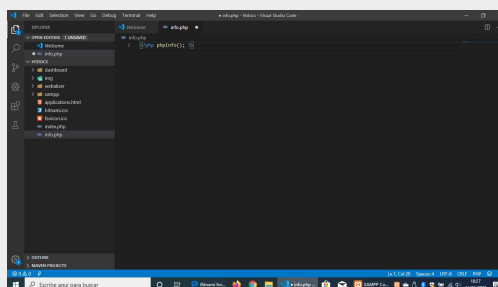


[Apache Friends \(GNU/GPL\)](#)

Abrimos `htdocs` con VSC, y creamos `info.php` (Paso 5)

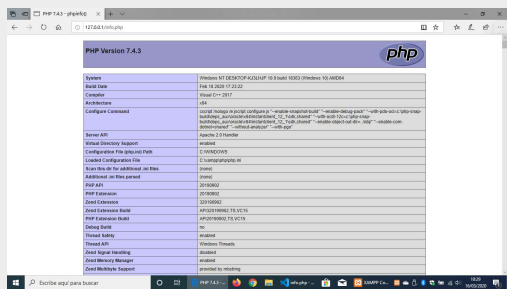


Captura de pantalla de Microsoft Windows (Elaboración propia)



Captura de pantalla de Microsoft Windows (Elaboración propia)

Vemos el resultado en el Navegador



Captura de pantalla de Mozilla Firefox (Elaboración propia)

- 1
- 2
- 3
- 4
- 5
- 6

3.3.- Programación web con Java.

Java es el lenguaje de programación más utilizado hoy en día. Es un lenguaje orientado a objetos, basado en la sintaxis de C y C++ y eliminando algunas características de éstos que daban lugar a errores de programación, como los punteros. Todo el código que escribas en Java debe pertenecer a una clase.

El código fuente se escribe en archivos con extensión `.java`. El compilador genera por cada clase un archivo `.class`. Para ejecutar una aplicación programada en Java necesitamos tener instalado un entorno de ejecución (JRE). Para crear aplicaciones en Java necesitamos el kit de desarrollo de Java (JDK), que incluye el compilador.

Como ya viste, existen básicamente dos tecnologías que te permiten programar páginas web dinámicas utilizando Java EE : `📁` `servlets` y `JSP` (páginas web que contienen instrucciones para añadir contenido de forma dinámica).

Aunque no es así en todos los casos, la mayoría de implementaciones disponibles para `JSP` compilan cada página y generan un `📁` `servlet` a partir de la misma la primera vez que se va a ejecutar. Este `📁` `servlet` se almacena para ser usado en futuras peticiones.

Por ejemplo, si quieres calcular la suma de dos números y enviar el resultado al navegador, lo podríamos realizar con una página `JSP`, incluyendo el código en Java dentro de las etiquetas `HTML` utilizando los delimitadores `<%` y `%>` de la siguiente manera:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Prueba JSP</title>
    <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
  </head>
  <body>
    Prueba de página JSP <br>
    La suma de 2 y 3 es: <% out.println(2+3) %>
  </body>
</html>
```

O también utilizando el método `println` dentro de un `📁` `servlet` como el siguiente, que obtiene los valores a sumar de otra página:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class miServlet extends HttpServlet {

    public void doGet(HttpServletRequest req, HttpServletResponse res) throws ServletException
```



[Ordercrazy \(CC0\)](#)

```
res.setContentType("text/html");
PrintWriter out = res.getWriter();

int num1 = Integer.parseInt(req.getParameter("num1"));
int num2 = Integer.parseInt(req.getParameter("num2"));

out.println("<!DOCTYPE html>");
out.println("<html>");
out.println("  <head>");
out.println("    <title>Prueba de servlet</title>");
out.println("  </head>");
out.println("  <body>");
out.println("    Prueba de servlet Java.");
out.println("    La suma de " + num1 + " + " + num2 + " es: " + (num1 + num2));
out.println("  </body>");
out.println("</html>");
}
```

No hay nada que se pueda hacer con JSP que no pueda hacerse también con  servlets. De hecho, como ya viste, las primeras se suelen convertir en  servlets para ser ejecutadas.

El problema de utilizar  servlets directamente es que, aunque son muy eficientes, son muy tediosos de programar puesto que hay que generar la salida en código HTML con gran cantidad de funciones como `println`. Este problema se resuelve fácilmente utilizando JSP, puesto que aprovecha la eficiencia del código Java, para generar el contenido dinámico, y la lógica de presentación se realiza con HTML normal.

De esta forma estas dos tecnologías se suelen combinar para crear aplicaciones web. Los  servlets se encargan de procesar la información y obtener resultados, y las páginas JSP se encargan del interface, incluyendo los resultados obtenidos por los  servlets dentro de una página web.

3.4.- Programación web con PHP.

PHP es un lenguaje de guiones de propósito general, pero diseñado para el desarrollo de páginas web dinámicas utilizando código embebido dentro del lenguaje de marcas. Su sintaxis está basada en la de C / C++, y por lo tanto es muy similar a la de Java. Aunque lo puedes hacer de otras formas, los delimitadores recomendados para incluir código PHP dentro de una página web son `<?php` y `?>`.

 Imagen de un óvalo de fondo violeta y las letras php en el medio

[Colin Viebrock \(CC BY-SA\)](#)

El código se ejecuta por un entorno de ejecución con el que se integra el servidor web (normalmente utilizando Apache con el módulo `mod_php`). La configuración tanto del servidor web Apache, como de PHP, se realiza por medio de ficheros de configuración. El de Apache es `httpd.conf`, y el de PHP es `php.ini`. Este fichero, `php.ini`, puede encontrarse en distintas ubicaciones. La función `phpinfo()` que ejecutaste antes te informa, entre otras muchas cosas, del lugar en que se encuentra almacenado el fichero `php.ini` en tu ordenador. En nuestro caso es en `/etc/php/php7.x/apache2/php.ini`. (php7.x es la versión de PHP, en la fecha de elaboración de este manual y con Ubuntu 20.04 la 7.3)

Dependiendo de cómo se integre PHP con Apache, los cambios que realices en su fichero de configuración se aplicarán en un momento o en otro. Si como es nuestro caso, utilizamos `mod_php` para ejecutar PHP como un módulo de Apache, las opciones de configuración de PHP se aplicarán cada vez que se reinicie Apache. Por tanto, no te olvides de hacerlo cada vez que hagas cambios en `php.ini`. Por ejemplo: `sudo systemctl restart apache2`

Algunas de las directivas más utilizadas que figuran en el fichero `php.ini` son:

- ✔ **short_open_tags**. Indica si se pueden utilizar en PHP los delimitadores cortos `<?` y `?>`. Es preferible no usarlos, pues puede causarnos problemas si utilizamos páginas con XML. Para prohibir la utilización de estos delimitadores con PHP le asignamos a esta directiva el valor **Off**. Por defecto suelen estar a **On**.
- ✔ **max_execution_time**. Permite que puedas ajustar el número máximo de segundos que podrá durar la ejecución de un script PHP. Evita que el servidor se bloquee si se produce algún error en un script.
- ✔ **display_errors**. Permite visualizar los errores que se produzcan en el código PHP. Para ver el nivel de detalles de los errores mostrados se complementa con la directiva siguiente, los valores recomendados por defecto son: Para un entorno en producción a **Off**, para un entorno en desarrollo a **On**. Por defecto suele estar en **On**.
- ✔ **error_reporting**. Indica qué tipo de errores se mostrarán en el caso de que se produzcan. Por ejemplo, si haces `error_reporting = E_ALL`, te mostrará todos los tipos de errores. Si no quieres que te muestre los avisos pero sí otros tipos de errores, puedes hacer `error_reporting = E_ALL & ~E_NOTICE`.
- ✔ **file_uploads**. Indica si se pueden o no subir ficheros al servidor por HTTP.
- ✔ **upload_max_filesize**. En caso de que se puedan subir ficheros por HTTP, puedes indicar el límite máximo permitido para el tamaño de cada archivo. Por ejemplo, `upload_max_filesize = 1M`.

- ✓ **post_max_size.** Complementa la directiva anterior, establece el tamaño máximo de un archivo subido por POST, Por ejemplo, `post_max_size = 1M`.

Iremos viendo otras directivas a lo largo del módulo cuando las vayamos necesitando. Realiza la siguiente actividad para comprobar si recuerdas las que acabamos de ver.



Actividad de repaso

Responde verdadero o falso a los siguientes supuestos.

Caso 1: Para mi proyecto necesito subir pdf de hasta 5M de tamaño. Las subidas serán todas por formularios (POST), he revisado la directiva siguiente y su valor, siendo esta: `upload_max_filesize = 15M`.

¿Podré subir mis archivos?

 Sugerencia

☐ Verdadero ☐ Falso

Falso

No es suficiente con la directiva anterior, necesitamos establecer el valor de la directiva `post_max_size` a 5M como mínimo y `file_uploads` a **On**.

Caso 2: En un un archivo php que debo modificar he visto el código siguiente:

```
<?
    $var=45;
    echo $var;
?>
```

El valor de la directiva `short_open_tags` es **Off**.

¿Funcionará dicho código?

☐ Verdadero ☐ Falso

Falso

Para que el código funcione el valor de la directiva `short_open_tags`, debe ser **On**. Aunque no es recomendable.

Caso 3: Acabo de cambiar mi proyecto PHP, de producción a desarrollo, el valor de la directiva `display_errors` es el que viene por defecto.

¿Lo he hecho bien?

☐ Verdadero ☐ Falso

Falso

Por defecto el valor de la directiva `display_errors` es **On** que no se recomienda para un entorno en producción, solo para entornos en desarrollo. Lo correcto hubiese sido ponerlo a **Off**.



Para saber más

En la documentación de PHP se incluye una lista completa de las directivas que se pueden utilizar en `php.ini`.

[Lista completa de las directivas](#)

A medida que vayas escribiendo código en PHP, será útil que introduzcas en el mismo algunos comentarios que ayuden a revisarlo cuando lo necesites. En una página web los comentarios al HTML van entre los delimitadores `<!--` y `-->`. Dentro del código PHP, hay tres formas de poner comentarios:

- ✓ Comentarios de una línea utilizando `//`. Son comentarios al estilo del lenguaje C. Cuando una línea comienza por los símbolos `//`, toda ella se considera que contiene un comentario, hasta la siguiente línea.
- ✓ Comentarios de una línea utilizando `#`. Son similares a los anteriores, pero utilizando la sintaxis de los scripts de Linux.
- ✓ Comentarios de varias líneas. También iguales a los del lenguaje C. Cuando en una línea aparezcan los caracteres `/*`, se considera que ahí comienza un comentario. El comentario puede extenderse varias líneas, y acabará cuando escribas la combinación de caracteres opuesta: `*/`.

Recuerda que cuando pongas comentarios al código PHP, éstos no figurarán en ningún caso en la página web que se envía al navegador (justo al contrario de lo que sucede con los comentarios a las etiquetas HTML).



Autoevaluación

Relaciona cada parámetro de la configuración de PHP con su finalidad:

Ejercicio de relacionar

Parámetro	Relación	Finalidad
<code>file_uploads</code>		1. Indica si se pueden subir ficheros al servidor web.

<code>max_execution_time</code>	☐	2. Indica qué tipo de delimitadores se pueden usar para el código PHP.
<code>upload_max_filesize</code>	☐	3. Limita el tamaño máximo de los ficheros que se suben al servidor.
<code>short_open_tags</code>	☐	4. Limita el tiempo máximo de ejecución de un guión PHP.

Enviar

Hay muchos parámetros de configuración en `php.ini`. Éstos son solo algunos de los más importantes.

3.4.1.- Variables y tipos de datos en PHP.

Como en todos los lenguajes de programación, en PHP puedes crear variables para almacenar valores. Las variables en PHP siempre deben comenzar por el signo \$. Los nombres de las variables deben comenzar por letras o por el carácter `_`, y pueden contener también números. Sin embargo, al contrario que en muchos otros lenguajes, en PHP no es necesario declarar una variable ni especificarle un tipo (entero, cadena,...) concreto. Para empezar a usar una variable, simplemente asignarle un valor utilizando el operador `=`.

```
$mi_variable = 7;
```



[Staecker](#) (Dominio público)

Dependiendo del valor que se le asigne, a la variable se le aplica un tipo de datos, que puede cambiar si cambia su contenido. Esto es, el tipo de la variable se decide en función del contexto en que se emplee.

```
// Al asignarle el valor 7, la variable es de tipo "entero"
$mi_variable = 7;

// Si le cambiamos el contenido
$mi_variable = "siete";
// La variable puede cambiar de tipo
// En este caso pasa a ser de tipo "cadena"
```

Los tipos de datos simples en PHP son:

- ✔ **booleano** (boolean). Sus posibles valores son `true` y `false`. Además, cualquier número entero se considera como `true`, salvo el `0` que es `false`.
- ✔ **entero** (integer). Cualquier número sin decimales. Se pueden representar en formato decimal, octal (comenzando por un `0`), o hexadecimal (comenzando por `0x`).
- ✔ **real** (float). Cualquier número con decimales. Se pueden representar también en notación científica.
- ✔ **cadena** (string). Conjuntos de caracteres delimitados por comillas simples o dobles.
- ✔ **null**. Es un tipo de datos especial, que se usa para indicar que la variable no tiene valor.

Por ejemplo:

```
$mi_booleano = false;
$mi_entero = 0x2A;
$mi_real = 7.3e-1;
$mi_cadena = "texto";
$mi_variable = Null;
```

Si realizas una operación con variables de distintos tipos, ambas se convierten primero a un tipo común. Por ejemplo, si sumas un entero con un real, el entero se convierte a real antes de realizar la suma:

```
$mi_entero = 3;
$mi_real = 2.3;
$resultado = $mi_entero + $mi_real;
// La variable $resultado es de tipo real
```

Estas conversiones de tipo, que en el ejemplo anterior se lleva a cabo de forma automática, también se pueden realizar de forma forzada:

```
$mi_entero = 3;
$mi_real = 2.3;
$resultado = $mi_entero + (int) $mi_real;
// La variable $mi_real se convierte a entero (valor 2) antes de sumarse.
// La variable $resultado es de tipo entero (valor 5)
```

Por otra parte tenemos unas funciones para comprobar el tipo de dato como:

- ✔ `is_bool()`: Comprueba si una variable es de tipo `booleano`.
- ✔ `is_float()`: Comprueba si el tipo de una variable es `float`.
- ✔ `is_numeric()`: Comprueba si una variable es un número o un `string` numérico.
- ✔ `is_string()`: Comprueba si una variable es de tipo `string`.
- ✔ `is_array()`: Comprueba si una variable es un `array`.
- ✔ `is_object()`: Comprueba si una variable es un objeto.

Todas devuelven `true` o `false`, por ejemplo:

```
$var=23; is_int($var); //devuelve true
is_int(23); //devuelve true
is_int('23'); //devolverá false
```



Debes conocer

En la documentación de PHP se especifican las conversiones de tipo posibles y los resultados obtenidos con cada una:

[Conversiones de tipos posibles y los resultados obtenidos](#)

3.4.2.- Expresiones y operadores.

Como en muchos otros lenguajes, en PHP se utilizan las expresiones para realizar acciones dentro de un programa. Todas las expresiones deben contener al menos un operando y un operador. Por ejemplo:

```
$mi_variable = 7;
$a = $b + $c;
$valor++;
$x += 5;
```



Edson Suares (CC BY-NC)

Los operadores que puedes usar en PHP son similares a los de muchos otros lenguajes como Java. Entre ellos tenemos operadores para:

- ✔ **Realizar operaciones aritméticas:** negación, suma, resta, multiplicación, división y módulo. Entre éstos se incluyen operadores de pre y post incremento y decremento, ++ y --.

Estos operadores incrementan o decrementan el valor del operando al que se aplican. Si se utilizan junto a una expresión de asignación, modifican el operando antes o después de la asignación en función de su posición (antes o después) con respecto al operando. Por ejemplo:

```
$a = 5;
$b = ++$a;
// Antes se le suma uno a $a (pasa a tener valor 6)
// y después se asigna a $b (que también acaba con valor 6).

$a = 5;
$b = $a++;
// Antes se asigna a $b el valor de $a (5).
// y después se le suma uno a $a (pasa a tener valor 6)
```

- ✔ **Realizar asignaciones.** Además del operador =, existen operadores con los que realizar operaciones y asignaciones en un único paso (+=, -=,...).
- ✔ **Comparar operandos.** Además de los que nos podemos encontrar en otros lenguajes (>, >=,...), en PHP tenemos dos operadores para comprobar igualdad (==, ===) y tres para comprobar diferencia (<>, != y !==).

Los operadores <> y !== son equivalentes. Comparan los valores de los operandos. El operador === devuelve verdadero (true) sólo si los operandos son del mismo tipo y además tienen el mismo valor. El operador !== devuelve verdadero (true) si los valores de los operandos son distintos o bien si éstos no son del mismo tipo. Por ejemplo:

```
$x = 0;
// La expresión $x == false es cierta (devuelve true).
```

```
// Sin embargo, $x === false no es cierta (devuelve false) pues $x es de tipo entero, n
```

- ✔ **Comparar expresiones booleanas.** Tratan a los operandos como variables booleanas (`true` o `false`). Existen operadores para realizar un **y lógico** (operadores `and` o `&&`), **o lógico** (operadores `or` o `||`), **No lógico** (operador `!`) y **o lógico exclusivo** (operador `xor`).
- ✔ **Realizar operaciones con los bits que forman un entero:** invertirlos, desplazarlos, etc.



Debes conocer

En la documentación de PHP se explican en detalle todos los operadores disponibles en el lenguaje y la forma en que se utilizan:

[Operadores disponibles en el lenguaje](#)

3.4.3.- Ámbito de utilización de las variables.

En PHP puedes utilizar variables en cualquier lugar de un programa. Si esa variable aún no existe, la primera vez que se utiliza se reserva espacio para ella. En ese momento, dependiendo del lugar del código en que aparezca, se decide desde qué partes del programa se podrá utilizar esa variable. A esto se le llama visibilidad de la variable.

Si la variable aparece por primera vez dentro de una función, se dice que esa variable es local a la función. Si aparece una asignación fuera de la función, se le considerará una variable distinta. Por ejemplo:

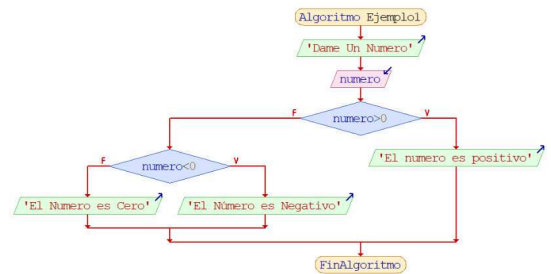


Diagrama realizado con Dia (Elaboración propia)

```

$a = 1;

function prueba()
{
    // Dentro de la función no se tiene acceso a la variable $a anterior
    $b = $a;

    // Por tanto, la variable $a usada en la asignación anterior es una variable nueva
    // que no tiene valor asignado (su valor es null)
}
  
```

Si en la función anterior quisieras utilizar la variable **\$a** externa, podrías hacerlo utilizando la palabra global. De esta forma le dices a PHP que no cree una nueva variable local, sino que utilice la ya existente.

```

$a = 1;

function prueba()
{
    global $a;
    $b = $a;
    // En este caso se le asigna a $b el valor 1
}
  
```

Para trabajar con variable globales también podemos usar el array asociativo: **\$GLOBALS**.

```

$a = 23;

function prueba()
{
  
```

```
$a=50;  
  
echo $a . "<br>"; //mostrará el valor local de a, es decir 50  
echo $GLOBALS["a"]; //mostrará el valor global de a, es decir 23  
}
```

Las variables locales a una función desaparecen cuando acaba la función y su valor se pierde. Si quisieras mantener el valor de una variable local entre distintas llamadas a la función, deberás declarar la variable como estática utilizando la palabra **static**.

```
function contador()  
{  
    static $a=0;  
  
    $a++;  
    // Cada vez que se ejecuta la función, se incrementa el valor de $a  
}
```

Las variables estáticas deben inicializarse en la misma sentencia en que se declaran como estáticas. De esta forma, se inicializan sólo la primera vez que se llama a la función.



Para saber más

Puedes consultar más ejemplos sobre los ámbitos de las variables en la documentación de PHP.

[Ámbito de las variables](#)



Autoevaluación

Si hacemos `$a=1`, ¿cuál de las siguientes comparaciones es verdadera?

- ☐ `"1" === $a`
- ☐ `$a == false`
- ☐ `a$++ == 2`
- ☐ `--$a == false`

No es correcta. El operador `===` compara valores y tipos, y aunque el valor de ambos operandos es el mismo, `$a` es una variable de tipo entero, y `"1"` es una cadena.

No es correcta porque en PHP cualquier número entero, incluido el uno, se considera true.

No es correcta porque primero se realiza la comparación y después se incrementa el valor de `$a`. Por tanto, el valor de `$a` que se utiliza en la comparación es 1.

Efectivamente es correcto. Antes de hacer la comparación, se decrementa `$a`, con lo cual su valor pasa a ser cero. Y en PHP, cero es equivalente a false.

Solución

1. Incorrecto
2. Incorrecto
3. Incorrecto
4. Opción correcta



Autoevaluación

Para cada uno de los supuestos siguientes marcaremos verdadero o falso.

Caso 1

```
$nombre="Juan";

function saludo(){
    echo "Hola $nombre<br />";
}

saludo();
```

El anterior código nos mostrará "Juan"

☐ Verdadero ☐ Falso

Falso

Incorrecto la variable **\$nombre** no existe en la función **saludo()**

Caso 2

```
$nombre="Juan";

function saludo(){
    $nombre="Pedro";
    echo "Hola $nombre<br />";
}

saludo();
```

El anterior código nos mostrará "Pedro"

☐ Verdadero ☐ Falso

Verdadero

El ámbito de la variable **\$nombre** en la función **saludo()** es local, luego efectivamente **\$nombre="Pedro"**

Caso 3

```
$nombre="Juan";

function saludo(){
    global $nombre;
    $nombre="Ana";
    echo "Hola $nombre<br />";
}

saludo();
echo $nombre."<br />";
```

El anterior código nos mostrará "Ana" y "Juan"

☐ Verdadero ☐ Falso

Falso

El código nos mostrará dos veces Ana, pues al usar **global \$nombre** en la función **saludo()** hemos modificado su valor global

